

Quant From Zero

Learning quantitative finance by dissecting a working engine

```
git clone https://github.com/Lukey-7/AxiomQuant
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build --parallel
./build/bin/axiomquant --data sample_data --no-db
```

A companion to the AxiomQuant repository: what quantitative finance is, how a backtest lies to you, what every module of this engine does, how to run it, and eight experiments that show each idea in action.

Part 1 — What quantitative finance actually is

1.1 The one-sentence version

Quantitative finance is using data and maths to decide what to buy and sell, how much of it, and how much you might lose — instead of deciding by opinion.

That is the whole field. Everything else is detail about *which* data, *which* maths, and *how honestly* you check yourself.

1.2 What a "strategy" is

A strategy is a rule that turns information into positions. Two examples this engine implements:

• **Moving-average crossover.** "If the average price over the last 20 days rises above the average over the last 50 days, buy. When it falls back below, sell." A trend-following rule.

• **Cross-sectional momentum.** "Every 20 days, rank all the stocks I can hold by their return over the last 60 days, hold the top 2, sell everything else." A rotation rule.

Notice what a rule is not: it is not a prediction of the future, and it is not a feeling about a company. It is a mechanical procedure. That is exactly why it can be tested.

1.3 What "edge" means, and why it is rare

An **edge** is a reason your rule should make money that survives after costs, and that is not simply a repackaging of the market going up. That last clause is the one beginners miss.

If stocks rise 10% a year and your strategy makes 10% a year, you have no edge — you have the market. The question is always *compared to what?* In this project the comparison is buy-and-hold: put the money in an index fund on day one and do nothing. Any strategy that cannot beat that, after costs, is a more expensive way of doing nothing.

Most published-looking strategies have no edge. That is not cynicism; it is what happens when thousands of people test millions of rules on the same few decades of data. Part 3 explains the mechanisms by which a strategy *appears* to have an edge when it does not, and Part 7 shows what this engine found when it tested honestly.

1.4 Who does this for a living

Roughly four roles, and they use the same vocabulary:

• **Quant researcher / strategist.** Invents and tests rules. Lives in data, statistics and doubt.

• **Quant developer.** Builds the machinery — backtesters, data pipelines, execution systems. This project is mostly quant-developer work with a researcher's questions attached.

• **Risk quant.** Answers "how much can we lose, and how likely is that?" Value-at-Risk, stress tests.

• **Execution / trading.** Gets orders into the market without moving the price against you. This is where "transaction costs" stop being a formula and become a job.

If you are learning to get hired, the quant-developer path is the most accessible from programming, and this repository is deliberately shaped like that work: correct machinery, tested, with results reported honestly rather than flatteringly.

Part 2 — The vocabulary, in plain words

You need about fifteen terms. Each one below has the plain meaning, the formula, and where the engine computes it, so you can read the code next to the definition.

2.1 Price, return, and why we use returns

A **price** is what one share costs. A **return** is the percentage change between two prices:

```
return on day t = price[t] / price[t-1] - 1
```

A stock going from \$100 to \$105 has a return of 0.05, or 5%. We work with returns rather than prices because returns are comparable across assets: a 1% move means the same thing for a \$10 stock and a \$4,000 one.

Log returns ($\ln(\text{price}[t] / \text{price}[t-1])$) show up in volatility maths because they add up over time instead of compounding, which makes the algebra easier. The engine uses simple returns for performance and log returns for volatility; both appear in `risk/` and `indicators/`.

Adjusted prices. If a company pays a dividend or splits its shares, the raw price drops without anybody losing money. Adjusted prices remove those artefacts. `scripts/fetch_data.py` applies the adjustment factor to open, high, low and close together, so a bar stays internally consistent.

2.2 Volatility — how bumpy the ride is

Volatility is the standard deviation of returns: the typical size of a daily move, ignoring direction. Because a year has about 252 trading days, we annualise:

```
annual volatility = standard deviation of daily returns × √252
```

An annual volatility of 20% means a typical year's movement is about $\pm 20\%$. The $\sqrt{252}$ comes from randomness accumulating with the square root of time, not linearly.

Where: `risk/src/metrics.cpp` (`annualized_volatility`), and the rolling version in `indicators/include/quant/indicators/rolling_volatility.hpp`.

2.3 Sharpe ratio — return per unit of worry

The single most-quoted number in the field:

```
Sharpe = (annual return - risk-free rate) / annual volatility
```

It answers: for each unit of bumpiness you endured, how much did you earn above just holding cash? The **risk-free rate** is what you would earn with no risk (a government bill); this project uses 2%.

Rules of thumb: below 0.5 is unimpressive, around 1 is good, above 2 for a long period is rare enough that you should suspect a mistake. A Sharpe quoted without an error bar is nearly meaningless — see §3.5 and Part 7, where this project found a Sharpe of 0.19 that is statistically indistinguishable from zero.

Sortino ratio is the same idea but divides by downside volatility only, on the argument that upside surprises are not "risk".

2.4 Drawdown — the number that actually hurts

Maximum drawdown is the worst peak-to-trough fall in your account:

```
drawdown[t] = (highest equity so far - equity[t]) / highest equity so far  
max drawdown = the largest of those
```

If you started with \$100k, rose to \$150k, and fell to \$90k before recovering, your maximum drawdown is 40%. This is the number that causes people to abandon a strategy at the worst possible moment. Volatility is what a strategy does on average; drawdown is what it does to you.

Where: `risk/src/metrics.cpp` (`drawdown_info`), which also reports how long you stayed underwater.

2.5 VaR and CVaR — the tail

Value-at-Risk (95%) is the loss you should exceed only one day in twenty. **CVaR** (also called expected shortfall) is the average loss on those bad days — a more honest number, because VaR tells you where the cliff starts and CVaR tells you how far down the rocks are.

This project computes three versions (historical, Gaussian, and Cornish-Fisher, which corrects for markets having fatter tails than a bell curve) in `risk/src/var_cvar.cpp`.

2.6 Correlation and covariance — how things move together

Correlation runs from -1 (perfect opposites) to $+1$ (identical moves). **Covariance** is the same idea unscaled, and it is what portfolio maths actually needs.

This matters because risk does not add up simply. Two assets with 20% volatility each, held half and half, give you 20% volatility only if they move in lockstep; if they are uncorrelated you get about 14%. That reduction is diversification, and it is the closest thing to a free lunch in finance.

Where: `optimization/src/portfolio_stats.cpp`.

2.7 Alpha and beta

Beta is how much of your return is just the market moving (multiplied by your exposure to it). **Alpha** is what is left over — the part attributable to skill. Most apparent alpha, measured carelessly, turns out to be beta in disguise: Part 7 shows a strategy whose "edge" was mostly being long an index during a bull market.

2.8 Transaction costs — where paper profits go to die

Four separate leaks, all modelled in `backtest/include/quant/backtest/execution_model.hpp`:

Cost	What it is	This project's default
Commission	The broker's fee	\$0.005/share, \$1 minimum, plus 1 basis point of value
Spread	Buy price is above sell price; you cross half of it	3 bp total, charged as 1.5 bp
Slippage	The price moves between decision and fill	2 bp
Market impact	Your own order pushes the price	$0.1 \times \sqrt{(\text{your size} / \text{volume})}$

A **basis point** (bp) is 0.01%. These sound tiny. A strategy trading 500 times a year pays them 500 times, which is why §3.4 and the cost-sensitivity sweep exist.

Part 3 — How a backtest works, and the four ways it lies

3.1 The mechanics

A **backtest** replays history one bar (one day) at a time:

1. Show the strategy today's data.
2. Let it emit orders.

3. Fill those orders at tomorrow's opening price, charging costs.
4. Update the portfolio, record the equity.
5. Repeat for every day in the sample.

That loop is `backtest/src/engine.cpp`, and it is only about 200 lines. The difficulty in backtesting is not the loop; it is everything that quietly makes the answer wrong.

3.2 Lie #1 — look-ahead bias

What it is. Using information you could not have had at the moment you traded. The classic version: your rule uses today's closing price, and your backtest also **buys at today's close**. Nobody can do that — once the close is printed, the close is gone.

Why it matters. It manufactures returns that evaporate in live trading, and it is the single most common reason a backtest looks good and a real account does not.

What this project does. Orders emitted on day **t** are filled at day **t+1**'s open, always. The alternative is available only so the bias can be **measured**: `-fill close` runs the cheating version, and the CLI prints both side by side. On real data the cheat is worth up to 0.03 of Sharpe here — small, because these are daily signals on liquid stocks, but it is measured rather than assumed.

3.3 Lie #2 — survivorship bias

What it is. Testing on the companies that exist today. Lehman Brothers is not in today's S&P 500; neither is Kodak. A backtest that ranks "the S&P 500" using today's list is silently ranking the survivors, which is knowledge from the future.

Why it matters. It is the largest single distortion this project measured. Same strategy, same code, same dates:

Universe	Return	Sharpe	Max drawdown
5 symbols chosen in 2026	+957.7%	0.88	38.9%
The S&P 500 as it actually was	+106.6%	0.31	75.6%

What this project does. `data/src/membership.cpp` holds a calendar of which symbol belonged to the index between which dates, built from a public record of index changes. The momentum strategy may only rank symbols that were members **on that date**. Full write-up: [reports/SP500_STUDY.md](https://axiomquant.com/reports/SP500_STUDY.md).

3.4 Lie #3 — costs assumed away

What it is. Reporting returns with optimistic or zero trading costs.

What this project does. Costs are on by default, and the CLI re-runs every strategy with all cost components scaled from $0\times$ to $10\times$, reporting the multiplier at which each strategy's profit reaches zero. On the point-in-time universe, momentum breaks even at $3.7\times$ — meaning its apparent profit depends on the cost model being accurate within a factor of four. That is fragile, and you only learn it by sweeping.

3.5 Lie #4 — overfitting, a.k.a. torturing the data

What it is. Trying many rules, keeping the best, and reporting its result as if you had only tried one. If you test 41 parameter pairs on random noise, the best of them will look good — that is arithmetic, not skill.

What this project does, three ways:

1. **Walk-forward testing.** Pick parameters using only the first two years, then trade them

unchanged on the next six months, which the search never saw. Roll forward and repeat. The in-sample Sharpe averaged 1.03; out of sample the same choices delivered 0.38.

2. **Deflated Sharpe ratio.** Ask how good the best of N tries would look *if all of them were worthless*, and check whether the winner clears that bar. Because 41 variations of one rule are not 41 independent attempts, the engine also estimates how correlated they are and counts them as ~8 effective trials.

3. **Bootstrap error bars.** Resample the out-of-sample returns in blocks thousands of times to get a confidence interval and a p-value. This is what turned "the strategy made 0.19 Sharpe" into "0.19, with a 95% interval of -0.42 to 0.82 , which is indistinguishable from zero".

3.6 The honest-backtest checklist

Borrow this for any backtest you meet, including your own:

• [] Are fills at a price available *after* the signal?

• [] Is the universe point-in-time, or today's survivors?

Q: Are realistic costs charged on every fill, and is the result robust to them being wrong?

⌘ Were parameters chosen on data the result was not measured on?

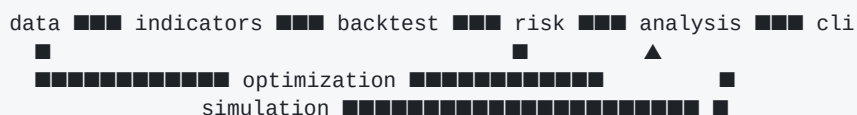
• [] Is there an error bar, and does the result survive it?

• [] Is the benchmark the honest one (buy-and-hold), not cash?

This engine answers all six in its normal output. Most backtests you will see online answer none.

Part 4 — A tour of the robot

Ten modules. For each: what it does, why it has to exist, and what to open first. The dependency order is the reading order — each module only uses the ones above it.



4.1 data/ — getting prices into memory, correctly

Does. Reads CSV files of daily bars, stores them in a `TimeSeries`, and lines several symbols up onto one shared calendar (`MarketDataUniverse`). Also holds the point-in-time membership calendar and optional SQLite storage.

Why it exists. Two stocks do not trade on identical days — holidays differ, listings start and end. Something must decide what "day 412" means for a portfolio. Two policies are offered: **intersection** (only days everything traded) and **union** (every day anything traded, carrying the last known price forward), which is what a changing universe needs.

Open first. `data/include/quant/data/types.hpp` for the Bar and TimeSeries layout, then `data/src/universe.cpp` for the alignment logic. Note the columnar layout: separate arrays for open, high, low, close rather than an array of structs, which is friendlier to the CPU when you sweep over one field.

Noob-relevant detail. `MembershipCalendar::is_member(ticker, date)` is three lines of string comparison, and it is the difference between +958% and +107% in Part 7. Small code, big honesty.

4.2 indicators/ — turning prices into signals

Does. Moving averages (SMA, EMA), RSI, MACD, Bollinger Bands, rolling volatility, ATR.

Why it exists. A strategy needs a compressed view of recent history. An indicator is just a function from a price series to another series.

Open first. `indicators/include/quant/indicators/sma.hpp`. The moving average uses a rolling accumulator, so each new day costs one addition and one subtraction rather than re-summing the window — the difference between $O(N)$ and $O(N \times \text{window})$ over a long backtest.

Noob-relevant detail. Every indicator returns NaN during its warm-up instead of zero. Zero would silently look like a real value and generate fake signals on day one; NaN forces the caller to handle it. This is a small example of a large principle: make invalid states loud.

4.3 `backtest/` — the event loop and the cost model

Does. Replays the timeline, asks the strategy for orders, fills them at the next open, charges costs, tracks positions and equity.

Why it exists. This is the machine under test. Everything else is input or measurement.

Open first. `backtest/src/engine.cpp` (the loop), then `backtest/include/quant/backtest/execution_model.hpp` (the four costs from §2.8), then any strategy in `backtest/include/quant/backtest/strategies/`.

Noob-relevant detail. Strategies never touch cash or positions directly — they push `Order` objects into a list and the engine decides what is affordable. That separation is why the same strategy object can be run under different cost models and fill rules without modification, which is what makes the cost sweep and the look-ahead comparison possible.

The five strategies included: buy-and-hold, SMA crossover, RSI mean reversion, cross-sectional momentum, and volatility targeting.

4.4 `risk/` — measuring what happened

Does. Turns an equity curve into CAGR, volatility, Sharpe, Sortino, Calmar, drawdowns, VaR/CVaR, skew and kurtosis, plus the text reports the CLI prints.

Why it exists. "It made money" is not an answer. *How much risk did it take to make it, and what did the bad days look like?* is.

Open first. `risk/src/metrics.cpp`. Every formula from Part 2 is here in a few lines each.

Noob-relevant detail. Trade statistics (win rate, profit factor) are computed from realised profit on closing fills, not from sale proceeds. Get that wrong and every sale looks like a win.

4.5 `simulation/` — what else could have happened

Does. Monte Carlo. Two engines: resampling historical returns (bootstrap), and simulating correlated asset paths from a fitted model (GBM). Reports the distribution of outcomes: median, percentiles, probability of loss, drawdown distribution.

Why it exists. A backtest gives you *one* history. The future will not be that history. Monte Carlo asks: given returns that behave like these, what is the spread of plausible outcomes?

Open first. `simulation/include/quant/simulation/rng.hpp` — a xoshiro256 generator seeded per path. Because each path has its own stream derived from (seed, path index), results are identical on 1 thread or 16, on Windows or Linux. Reproducibility is a design choice, not an accident.

Noob-relevant detail. Plain bootstrap assumes each day is independent, which is false: volatile days cluster. The block bootstrap (`-block`) resamples runs of consecutive days instead, preserving that clustering. The difference shows up mostly in drawdown statistics.

4.6 optimization/ — how much of each thing to hold

Does. Markowitz portfolio maths: expected returns, covariance (sample and Ledoit-Wolf shrinkage), minimum-variance and maximum-Sharpe portfolios, a constrained solver with weight caps, the efficient frontier, and risk parity.

Why it exists. Choosing *what* to hold is only half the problem; *how much* is the other half.

Open first. `optimization/src/constrained_qp.cpp`. The projection onto the bounded simplex — "find the nearest set of weights that sums to 1 and respects the caps" — is an exact sweep over breakpoints rather than an approximation, and the solver around it is accelerated projected gradient descent with adaptive restart.

Noob-relevant detail. Unconstrained mean-variance optimisation is famously unusable: it demands enormous long and short positions that swing wildly. Part 7 shows it reporting a Sharpe of -5.5 on 606 assets. The practical answers are constraints (weight caps, long-only) and shrinkage (pulling a noisy covariance matrix toward a simple structure), both implemented here.

4.7 analysis/ — the honesty layer

Does. Walk-forward evaluation, parameter sweeps, cost sensitivity, and the statistics: block bootstrap of the Sharpe ratio, probabilistic and deflated Sharpe, effective number of trials, automatic block-length selection.

Why it exists. Everything in Part 3 lives here. This module is the difference between a backtester and a research tool.

Open first. `analysis/src/walk_forward.cpp` for the train/test loop, then `analysis/src/significance.cpp` for the statistics.

Noob-relevant detail. `sweep_walk_forward` re-runs the entire protocol at several window sizes and in two modes, because a conclusion drawn from one split is one draw from a noisy process. If your result only holds for a 504/126 split, it is not a result.

4.8 cli/ — the eight-stage pipeline

Does. `axiomquant` loads data and runs eight stages: ingest, optional SQLite persistence, indicators, strategy tournament (plus the look-ahead and cost-sensitivity studies), in-sample parameter sweep with deflation, walk-forward with robustness and significance, portfolio optimisation, Monte Carlo. Two smaller binaries exist for CI: `axiom_optimizer_dump` (feeds the `cvxpy` cross-check) and `axiom_bench` (feeds the NumPy benchmark).

Open first. `cli/src/main.cpp`, and just read the `section(...)` calls to see the shape.

4.9 scripts/ — data and reporting, in Python

Script	Purpose
<code>fetch_data.py</code>	Get prices: live provider, your own CSVs, a Kaggle dataset, or a synthetic generator
<code>make_membership.py</code>	Turn an index-constituents file into the point-in-time calendar
<code>make_charts.py</code>	Render the CSV exports as SVG charts, standard library only
<code>check_optimizer.py</code>	Compare every optimizer against <code>cvxpy</code> and closed-form solutions
<code>benchmark.py</code>	Time the engine against vectorised NumPy doing the same work
<code>test_*.py</code>	Offline tests for the above

4.10 tests/ and CI — why you can trust any of it

64 tests run on every push across Linux (GCC), macOS (Clang), Windows (MSVC) and a sanitiser build that catches memory and undefined-behaviour bugs. Four more jobs check things tests cannot: the optimizer against an independent solver, performance against NumPy, formatting and static analysis, and test coverage (80%

floor).

Noob-relevant detail. The optimizer cross-check found a real bug: "risk parity" was normalising weights inside its iteration and converging to a portfolio whose risk contributions were **not** equal — off by up to 14 percentage points. Tests said it converged; only comparing against an independent solver revealed it was converging to the wrong thing. That is the argument for cross-validation in one story.

Part 5 — Running it yourself

5.1 What you need

Tool	Why	Windows install
C++20 compiler, CMake, Ninja	Build the engine	<code>winget install -e -id MSYS2.MSYS2</code> , then the pacman line below
Python 3.10+	Data and reporting scripts	Already on most machines
Git and GitHub CLI	Clone, and run the workflows	<code>winget install Git.Git</code> <code>GitHub.cli</code>

```
winget install -e --id MSYS2.MSYS2
C:\msys64\usr\bin\bash.exe -lc "pacman -S --needed --noconfirm mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-cmake mingw-w64-ucrt-x86_64-ninja mingw-w64-ucrt-x86_64-clang mingw-w64-ucrt-x86_64-clang-tools-extra"
```

Then add `C:\msys64\ucrt64\bin` to your PATH. On Linux or macOS, the system compiler and CMake are enough. No external C++ libraries are needed: Eigen and SQLite are vendored in `third_party/`.

5.2 Build and run

```
git clone https://github.com/Lukey-7/AxiomQuant && cd AxiomQuant
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel          # about a minute
./build/bin/quant_tests                 # 64 tests, under a second
./build/bin/axiomquant --data sample_data --no-db
```

The last command prints the whole eight-stage pipeline on the bundled synthetic data. Nothing touches the network and nothing is written unless you ask for it.

5.3 Every flag, and when you care

Flag	Default	Use it when
<code>-data <dir></code>	<code>sample_data</code>	Pointing at real prices you downloaded
<code>-ticker <symbol></code>	<code>SPY</code>	Single-name strategies should trade something else
<code>-capital <usd></code>	<code>100000</code>	Position sizes and the minimum-commission effect change
<code>-rf <rate></code>	<code>0.02</code>	The risk-free rate in every Sharpe
<code>-fill <open\</code>	<code>close></code>	<code>open</code> You want to <i>*see*</i> look-ahead bias (§3.2)

Flag	Default	Use it when
<code>-max-weight <w></code>	0.40	Changing the per-asset cap in the optimizer
<code>-paths <n> / -horizon <days></code>	50000 / 252	Monte Carlo size and horizon
<code>-seed <n></code>	42	Checking results are not seed-specific
<code>-bootstrap <n></code>	10000	Trading accuracy of the p-values for speed
<code>-block <days></code>	1	Keeping volatility clustering in the Monte Carlo
<code>-members <csv></code>	none	Removing survivorship bias (§3.3)
<code>-align <mode></code>	intersection	The universe changes over time — use union
<code>-wf-train / -wf-test</code>	504 / 126	Changing the walk-forward window sizes
<code>-db <path> / -no-db</code>	on / —	Persisting runs to SQLite, or not
<code>-export-dir <dir></code>	none	You want CSVs to chart or inspect

5.4 Getting real data

```
# Live daily bars (no API key needed)
python3 scripts/fetch_data.py live --out real_data --tickers SPY AAPL MSFT --start 2015-01-01

# CSVs you already have, e.g. a Kaggle download; long or per-symbol format both work
python3 scripts/fetch_data.py import ~/Downloads/all_stocks_5yr.csv --out kaggle_data

# Let the script download the Kaggle dataset itself (needs a Kaggle API token)
python3 scripts/fetch_data.py kaggle camnugent/sandp500 --out kaggle_data

# Synthetic data with known properties, for testing the machinery
python3 scripts/fetch_data.py synthetic --out synth_data --tickers AAA BBB CCC --seed 7

./build/bin/axiomquant --data real_data --no-db --export-dir out
python3 scripts/make_charts.py --input out --output out/images
```

5.5 The three workflows

Run from the repository, or from the Actions tab on GitHub:

```
gh workflow run "Real data"          # the standard study on live prices, ~2 minutes
gh workflow run "S&P 500 study"      # point-in-time universe, ~11 minutes, 771 symbols
```

CI runs automatically on every push. Each workflow uploads its prices, log and CSV exports as artifacts, which is how every number in the reports can be traced to a specific run.

5.6 Reading the output

The eight stages, and the one question each answers:

Stage	Question
1 Ingest	How many days and symbols line up?
2 SQLite	(Optional) persist this run
3 Indicators	What do the signals say about the latest bar?
4 Tournament	How did each strategy do under identical rules — plus look-ahead and cost sweeps
5 Sweep	What does the best parameter pair look like in-sample, and does it survive deflation?
6 Walk-forward	What happened out of sample, across window sizes, and is it distinguishable from luck?
7 Optimization	How much of each asset, under several objectives?
8 Monte Carlo	What is the distribution of next year's outcomes?

Start at stage 6. Stages 4 and 5 are the seductive ones; stage 6 is the one that tells the truth.

Part 6 — Dissect the robot: eight experiments

Each one is a single command, a prediction, and what the machine actually reports. Run them in order; they build on each other. Use `-paths 2000 -bootstrap 2000` to keep runs quick while you are exploring.

Experiment 1 — Make look-ahead bias visible

```
./build/bin/axiomquant --data sample_data --no-db --fill close
```

Predict: the cheating version (filling at the close that generated the signal) should look better. **Observe:** the "COST OF LOOK-AHEAD" table in stage 4 prints both. The gap is small on daily data, which is itself the lesson: look-ahead is deadly on intraday signals and modest on slow ones.

Experiment 2 — Find where the costs kill the strategy

```
./build/bin/axiomquant --data sample_data --no-db | sed -n '/COST SENSITIVITY/,/paying for/p'
```

Predict: strategies that trade more should decay faster as costs rise. **Observe:** buy-and-hold (1 trade) barely moves; momentum (hundreds of trades) collapses. The "break-even" column is the cost multiplier at which profit reaches zero — the margin of safety in your cost assumptions.

Experiment 3 — Watch in-sample results evaporate

```
./build/bin/axiomquant --data sample_data --no-db | sed -n '/\[5\8\]/,/\[7\8\]/p'
```

Predict: the best parameters in stage 5 will do worse in stage 6. **Observe:** the mean in-sample Sharpe against the mean out-of-sample Sharpe, and the decay between them. This one number is the entire argument for walk-forward testing.

Experiment 4 — Change the test and watch the answer move

```
./build/bin/axiomquant --data sample_data --no-db --wf-train 252 --wf-test 63
```

Predict: shorter windows mean more folds, less training data, noisier choices. **Observe:** the ROBUSTNESS table compares window sizes and both start modes (flat versus carried). If your conclusion changes between rows, you have learned something about your test, not about the market.

Experiment 5 — Turn survivorship bias on and off

```
gh workflow run "S&P 500 study" --ref main          # with the membership calendar
# then compare the momentum row against RESEARCH.md, which uses five symbols chosen today
```

Predict: picking winners from a list of known winners flatters the result. **Observe:** +957.7% becomes +106.6%, Sharpe 0.88 becomes 0.31, drawdown 38.9% becomes 75.6%. Largest effect in this entire project.

Experiment 6 — Ask whether the result is luck

```
./build/bin/axiomquant --data real_data --no-db --bootstrap 20000
```

Predict: a small positive Sharpe over a few years will not be distinguishable from zero. **Observe:** the interval and p-value under "IS THE OUT-OF-SAMPLE RESULT DISTINGUISHABLE FROM LUCK?". Notice that buy-and-hold's Sharpe *is* significant while the strategy's is not — the test has power, it simply is not finding an edge.

Experiment 7 — Break the Monte Carlo's independence assumption

```
./build/bin/axiomquant --data real_data --no-db --block 1    # days drawn independently
./build/bin/axiomquant --data real_data --no-db --block 10   # ten-day blocks
```

Predict: terminal wealth will barely change; drawdowns will get worse with blocks. **Observe:** exactly that. Bad days arriving together is what creates deep drawdowns, and i.i.d. resampling destroys that structure.

Experiment 8 — See mean-variance optimisation misbehave

```
./build/bin/axiomquant --data sample_data --no-db | sed -n '/MARKOWITZ/,/Legend/p'
```

Predict: the unconstrained portfolio will demand extreme long and short positions. **Observe:** unconstrained tangency wants large shorts; the constrained long-only version is sane. Then read §4.6 on what happens with 606 assets, where unconstrained optimisation produces a Sharpe of −5.5.

Going further. Add your own strategy: copy

backtest/include/quant/backtest/strategies/vol_target.hpp, change the rule inside on_bar, add it to the list in cli/src/main.cpp, rebuild, and it automatically appears in the tournament, the cost sweep and the look-ahead study.

Part 7 — What this project actually found

Every figure below is traceable to a named CI run; see [RESEARCH.md](#) and [reports/SP500_STUDY.md](#).

1. Simple technical strategies did not beat buy-and-hold. Over 11.7 years of real prices, the walk-forward SMA crossover returned +32.3% against buy-and-hold's +254.6%. It drew down less, but only because it spent most of a rising market in cash.

2. That result is not distinguishable from zero. Sharpe 0.19, 95% interval −0.42 to 0.82, $p = 0.28$. Buy-and-hold's 0.73 *is* significant ($p = 0.013$). Nearly twelve years of daily data pins a Sharpe to about ± 0.6 — worth remembering whenever you see one quoted to two decimals.

3. In-sample selection overstated performance by about 2.7×. Mean in-sample Sharpe 1.03; the same parameters delivered 0.38 out of sample.

4. Survivorship bias was worth roughly 850 percentage points. The momentum comparison in §3.3.

5. Volatility targeting was the only useful finding. Sizing a single index position by recent volatility matched buy-and-hold's risk-adjusted return (Sharpe 0.69 vs 0.71) while cutting the worst drawdown from 33.4% to 13.2%. No forecasting involved — just the fact that volatility persists.

6. Cross-checking found a real bug. The risk-parity portfolio was not equalising risk at all until an independent solver was put beside it in CI.

The meta-lesson: the machinery that makes a project *believable* — point-in-time data, error bars, cross-validation, cost sweeps — is most of the work, and it is what separates a toy backtester from research.

Part 8 — Where to go next

8.1 A learning path that matches this codebase

1. **Get comfortable with returns, volatility and Sharpe** (Part 2), then re-read stage 4's output until every column is obvious.
2. **Learn why backtests lie** (Part 3) and run experiments 1–5. This is the highest-value week you can spend; most people skip it and spend years on indicators instead.
3. **Learn enough statistics to defend a number:** sampling variation, confidence intervals, p-values, multiple-testing correction. Experiment 6 is the concrete version.
4. **Portfolio construction:** covariance, diversification, why unconstrained optimisation fails, shrinkage, risk parity. Experiment 8 plus §4.6.
5. **Market microstructure and execution:** spreads, order books, impact. This project models costs parametrically; real execution is a career.
6. **Then, if you still want them, machine-learning methods** — with the discipline from steps 2–3 already in place, because ML makes overfitting easier, not harder.

Useful books, in this order: *Trading and Exchanges* (Harris) for how markets work; *Advances in Financial Machine Learning* (López de Prado) for the deflated Sharpe ratio and the overfitting material implemented here; *Active Portfolio Management* (Grinold & Kahn) for the portfolio maths.

8.2 What is left to build here

Honest gaps, roughly in order of value:

• **One index, one regime.** 2015–2026 was kind to US large caps. Re-running from 2000 would cover a decade that was not.

• **166 delisted symbols still missing** from the point-in-time study, because free data sources drop them. A paid point-in-time database would close the last of the survivorship gap.

• **No shorting, no leverage, no position-level risk limits**, and no intraday data.

• **Momentum's own parameters were never tuned** on the honest universe; doing so would need the deflated Sharpe ratio to stay defensible.

• **Effective trials uses one average correlation;** clustering the trials would be sharper.

• **Execution realism:** partial fills, queue position, borrow costs, capacity limits.

Appendix A — Glossary

Term	Meaning
Alpha	Return attributable to skill, after removing market exposure
Basis point (bp)	0.01%
Beta	Sensitivity to the market's return
Bootstrap	Resampling your own data to estimate uncertainty
CAGR	Compound annual growth rate
Calmar	CAGR divided by maximum drawdown
Covariance	Unscaled measure of two series moving together
CVaR / Expected shortfall	Average loss on the worst days
Deflated Sharpe	Sharpe judged against what the best of N tries would show by luck
Drawdown	Fall from the highest equity so far
Equity curve	Account value over time
Fill	An executed order
Ledoit-Wolf shrinkage	Pulling a noisy covariance matrix toward a simple target
Look-ahead bias	Using information you could not have had
Monte Carlo	Simulating many possible futures
Out-of-sample	Data not used to choose parameters
Point-in-time	Data as it was known on the date, not as revised later
Risk parity	Weights that give each asset an equal share of portfolio risk
Sharpe ratio	Excess return divided by volatility
Slippage	Price movement between decision and fill
Survivorship bias	Testing only on the names that still exist
Volatility	Standard deviation of returns, usually annualised
Walk-forward	Choose parameters on a window, test on the next, roll forward

Appendix B — Formula sheet

return	$r[t] = P[t] / P[t-1] - 1$
log return	$= \ln(P[t] / P[t-1])$
annual volatility	$= \text{stdev}(r) \times \sqrt{252}$
CAGR	$= (\text{final} / \text{initial})^{(1/\text{years})} - 1$
Sharpe	$= (\text{annual return} - r_f) / \text{annual volatility}$
Sortino	$= (\text{annual return} - \text{MAR}) / \text{downside deviation}$
drawdown[t]	$= (\text{peak so far} - \text{equity}[t]) / \text{peak so far}$
Calmar	$= \text{CAGR} / \text{max drawdown}$
portfolio variance	$= w^T \Sigma w$
portfolio volatility	$= \sqrt{(w^T \Sigma w)}$
risk contribution of i	$= w[i] \times (\Sigma w)[i] / (w^T \Sigma w)$
VaR(95%) Gaussian	$= \text{mean} + z(0.05) \times \text{stdev}$
Cornish-Fisher z	$= z + (S/6)(z^2-1) + (K/24)(z^3-3z) - (S^2/36)(2z^3-5z)$
square-root impact	$= \Delta P/P = \eta \sqrt{(\text{order size} / \text{volume})}$
probabilistic Sharpe	$= \Phi((SR - SR^*) \sqrt{(T-1)} / \sqrt{(1 - \gamma^T SR + (\gamma^T - 1)/4 SR^2)})$

Appendix C — File map

Path	What lives there
data/	CSV loading, time series, universe alignment, membership calendar, SQLite
indicators/	SMA, EMA, RSI, MACD, Bollinger, rolling volatility, ATR
backtest/	Event loop, execution and cost model, portfolio, five strategies
risk/	Performance and risk metrics, VaR/CVaR, text reports
simulation/	Reproducible RNG, bootstrap and correlated GBM Monte Carlo
optimization/	Covariance and shrinkage, mean-variance solvers, frontier, risk parity
analysis/	Walk-forward, sweeps, cost sensitivity, bootstrap and deflated Sharpe
cli/	axiomquant pipeline, plus the benchmark and optimizer-dump tools
scripts/	Data fetching, membership building, charts, cross-check, benchmark
tests/	64 unit and regression tests
docs/	This guide, the study guide, the reports, the charts
.github/workflows/	CI, Real data, S&P 500 study
third_party/	Eigen and SQLite, vendored

This guide describes the repository as of the runs cited in Part 7. The commands are copy-pasteable; if one stops working, the workflows in `.github/workflows/` are the source of truth for how each study is run.